Paralelismo com Memória Compartilhada

Caick Wendell Lopes dos Santos, Camila De Araújo Bastos
Departamento de Tecnologia - Universidade Estadual de Feira de Santana (UEFS)
Av. Transnordestina, s/n, Novo Horizonte / Feira de Santana - BA, Brasil - 44036-9

1. Introdução

Este relatório técnico apresenta uma análise de desempenho e escalabilidade de diferentes abordagens de implementação do Conway's Game of Life (Jogo da Vida). O algoritmo consiste em um autômato celular bidimensional regido por regras de sobrevivência, morte e reprodução das células, determinadas pelo estado de sua vizinhança de Moore, composta pelas oito posições adjacentes de cada célula, formando uma vizinhança 3x3.

A estrutura de dados utilizada representa o universo por meio de uma matriz toroidal, na qual as bordas da matriz são conectadas continuamente por meio de operações de módulo aritmético. Essa abordagem permite simular um universo sem bordas, garantindo que as células localizadas nas extremidades também possuam uma vizinhança completa.

A evolução do sistema é realizada de forma síncrona, utilizando matrizes de estado temporárias. Dessa maneira, o novo estado de cada célula é calculado a partir do estado completo da geração anterior, evitando que células já atualizadas interfiram no cálculo das demais células da mesma geração.

2. Objetivo

A análise busca comparar as diferenças de tempo de execução, speedup, eficiência paralela, comportamento da hierarquia de memória e custos associados ao paralelismo em cada abordagem. Também são comparados os resultados das implementações em C, uma linguagem compilada, e Python, uma linguagem interpretada, considerando as diferenças de desempenho e os efeitos dessas características na execução do algoritmo em arquiteturas multicore.

Além disso, pretende-se avaliar o impacto do Global Interpreter Lock (GIL) na implementação com multithreading em Python, considerando sua influência sobre a execução concorrente de operações intensivas em CPU. A análise também compara essa abordagem com o multiprocessing, que utiliza processos independentes e permite explorar múltiplos núcleos de processamento sem a mesma limitação do GIL.
Por fim, são analisadas as diferenças entre os mecanismos de comunicação utilizados nas implementações com multiprocessing, comparando a comunicação por Pipe, que envolve a transferência de dados entre espaços de memória independentes.

3. Metodologia e Implementação

Para avaliar o impacto de diferentes estratégias de paralelização sobre o desempenho do Jogo da Vida de Conway, foram implementadas variantes do algoritmo, todas derivadas de uma mesma especificação de problema e de um mesmo baseline serial.

- C serial;
- C paralelizado com OpenMP;
- Python serial;
- Python com multithreading utilizando ThreadPoolExecutor;
- Python com multiprocessing utilizando comunicação por Pipe.

Todas as implementações compartilham a mesma condição inicial, as mesmas regras de transição e o mesmo critério de parada, garantindo a comparabilidade dos resultados. As versões diferem na linguagem utilizada, no modelo de execução e na forma como o trabalho é decomposto e sincronizado entre as unidades de execução concorrentes.

3.1. Definição do Problema e Parâmetros Comuns

Em todas as implementações, o universo é representado por uma grade toroidal (bordas conectadas por aritmética modular) de dimensões WxH, configuráveis por variáveis de ambiente (GOL_W, GOL_H), com valor padrão de 500x500 células. O estado inicial é determinístico e idêntico em todas as versões: uma cruz que atravessa o tabuleiro pela linha e pela coluna centrais (x=W/2 ou y=H/2), o que garante que qualquer divergência de resultado entre implementações seja atribuível à lógica de paralelização e não a diferenças de inicialização. O número de gerações simuladas

também é parametrizável (GOL_ITER, padrão 5000), executando-se sempre max_iter + 1 passos de evolução para manter paridade entre todas as versões.

As regras de transição seguem a formulação clássica do Jogo da Vida: para cada célula, contam-se os vizinhos vivos na vizinhança de Moore (oito células adjacentes, com vizinhança toroidal); uma célula viva permanece viva se possuir 2 ou 3 vizinhos vivos, e uma célula morta nasce se possuir exatamente 3 vizinhos vivos; qualquer outra condição resulta em morte ou permanência do estado morto.

Nas versões em C, os parâmetros w=500, h=500 e max_iter=5000 são fixados diretamente no código-fonte (em main), ao passo que as versões em Python os leem de variáveis de ambiente com os mesmos valores como padrão — diferença que não afeta a comparabilidade dos resultados, uma vez que ambas as linguagens são executadas com a mesma configuração de tamanho de grade e número de gerações.

Por fim, ao término de todas as iterações, o programa realiza a contagem total de células vivas no tabuleiro. Essa etapa final foi introduzida em todos os códigos estritamente para contornar as otimizações do compilador C (como a eliminação de código morto, ou dead code elimination): caso o estado final da matriz não fosse utilizado para produzir nenhuma saída, o compilador descartaria todo o processamento do laço principal, reduzindo o tempo de execução da simulação a zero.

3.2. Paralelização por Colunas

A estratégia de paralelização adotada baseou-se na divisão geométrica do tabuleiro em blocos de colunas, atendendo a um requisito explícito do trabalho. Do ponto de vista de arquitetura e desempenho, essa escolha traz implicações diretas para a localidade espacial de cache: como as matrizes em C e as listas nativas do Python adotam o armazenamento row-major (elementos contíguos dispostos linha por linha), o processamento restrito a fatias verticais estreitas impede o aproveitamento ideal das linhas de cache, o que eleva consideravelmente a taxa de cache misses. Apesar desse impacto na hierarquia de memória, a topologia por colunas simplifica o padrão de comunicação, pois restringe as áreas de fronteira (as "colunas fantasmas") estritamente às bordas laterais direita e esquerda, facilitando a troca de mensagens e a sincronização entre os processos ou threads vizinhos.

3.3. Implementação Serial em C (gol.c)

A versão serial em C constitui o baseline de mais baixo nível do trabalho, servindo de referência tanto para a versão paralela em OpenMP quanto para as reimplementações em Python. O universo é representado por uma matriz unsigned univ[h][w] alocada na pilha, e a função evolve segue exatamente a mesma estrutura de quatro laços aninhados descrita na versão Python: para cada célula (x,y), os vizinhos são contados por meio de dois laços internos que percorrem a vizinhança 3x3 com indexação toroidal via aritmética modular ((y1+h)%h, (x1+w)%w); o próprio valor da célula é descontado quando ela está viva, e o novo estado é escrito em uma matriz local new[h][w], também alocada na pilha, evitando a leitura de estados já atualizados da mesma geração. Uma segunda varredura completa copia new de volta para univ, reproduzindo o mesmo esquema de double buffering explícito da versão Python — de fato, a implementação em Python de gol.py foi construída como uma tradução direta desta versão em C, célula a célula e laço a laço, para servir de baseline funcionalmente equivalente entre as duas linguagens.

3.4. Implementação Paralela em C com OpenMP (gol_omp.c)

A versão gol_omp.c paraleliza a fase de evolução do algoritmo por meio de diretivas #pragma omp parallel for aplicadas ao laço externo sobre o eixo x (colunas), tanto na etapa de cálculo do próximo estado quanto na etapa de cópia de volta para univ e na inicialização do tabuleiro. Note-se que, em relação à versão serial, os laços de evolve são deliberadamente invertidos — o laço externo percorre x e o laço interno percorre y — de modo que a divisão de iterações feita automaticamente pelo OpenMP entre as threads ocorra por colunas e não por linhas, replicando a mesma decomposição unidimensional empregada manualmente nas versões multithread e multiprocess em Python.

Cada thread do OpenMP processa uma faixa contígua de colunas da matriz new de forma independente, sem necessidade de exclusão mútua, pois threads distintas nunca escrevem na mesma coluna. Como a matriz univ original permanece somente-leitura durante todo o cálculo da geração seguinte (a escrita ocorre exclusivamente em new), não há condição de corrida entre as threads na fase de cálculo; a barreira implícita ao final de cada região parallel for garante que todas as colunas de new estejam completas antes do início do laço de cópia paralela subsequente, que por sua vez

também está protegido por uma barreira implícita antes do avanço para a próxima geração. Diferentemente das versões multithread e multiprocess em Python, que precisam implementar manualmente a divisão de blocos, o balanceamento do resto da divisão e, no caso dos processos, a comunicação de bordas, o OpenMP delega ao runtime a criação das threads, a divisão do intervalo de iterações e a sincronização por barreiras implícitas, exigindo apenas a anotação dos laços paralelizáveis e a garantia de que os laços tenham iterações independentes.

3.5. Implementação Serial em Python (gol.py)

A versão serial estabelece o baseline de corretude e desempenho contra o qual as demais implementações são comparadas. A função evolve realiza, para cada geração, uma varredura completa da matriz WxH para cada célula, quatro laços aninhados percorrem a vizinhança 3x3 centrada na célula, contabilizando vizinhos vivos por meio de indexação modular ((y1+h)%h, (x1+w)%w), o que implementa a topologia toroidal sem necessidade de tratamento especial de bordas. O próprio valor da célula é descontado da contagem quando ela está viva, e o novo estado é escrito em uma matriz auxiliar (new), evitando que a leitura da vizinhança de uma célula seja corrompida pela escrita do novo estado de outra célula na mesma geração. Uma segunda passagem completa copia o estado calculated de volta para a matriz univ, caracterizando um esquema de double buffering explícito. A contagem final de células vivas (count_alive) é feita por soma sequencial simples, servindo de referência para validar a redução paralela nas demais versões.

3.6. Implementação com Multithreading em Python (gol_multithread.py)

Esta variante paraleliza a mesma lógica de evolução por meio de um ThreadPoolExecutor, decompondo o domínio em blocos contíguos de colunas — decomposição unidimensional análoga à empregada nas versões baseadas em processos. Como todas as threads compartilham o mesmo espaço de memória do processo, não há necessidade de comunicação explícita de bordas: cada thread lê diretamente das colunas vizinhas dentro da mesma matriz univ.

A evolução de cada geração é dividida em duas fases sincronizadas por barreiras implícitas de Future.result():

- Fase 1 (process_chunk): cada thread calcula o próximo estado de seu bloco de colunas [start_x, end_x) e escreve o resultado na matriz auxiliar new, sem jamais escrever em univ;
- Fase 2 (copy_chunk): somente após todas as threads da Fase 1 terminarem (garantido pelo bloqueio até a resolução de todos os Futures), cada thread copia seu bloco de new de volta para univ.

A separação estrita entre as duas fases evita condições de corrida: se a cópia ocorresse simultaneamente ao cálculo, uma thread poderia ler um estado parcialmente atualizado da geração seguinte ao processar a vizinhança de uma célula na borda de seu bloco. A contagem final de células vivas segue o mesmo princípio de decomposição por colunas, em que cada thread soma um bloco disjunto (count_chunk) e retorna o total parcial via valor de retorno do Future, evitando o uso de Lock para proteção de uma variável compartilhada — abordagem funcionalmente equivalente a uma cláusula reduction (+:total) do OpenMP.

Cabe notar que, por se tratar de paralelismo baseado em threads no CPython, o ganho de desempenho desta versão está sujeito às limitações impostas pelo Global Interpreter Lock (GIL), que serializa a execução de bytecode Python entre threads, restringindo o paralelismo efetivo majoritariamente aos períodos em que o interpretador libera o GIL.

3.7. Implementação com Multiprocessamento em Python e Troca de Bordas via Pipe (gol_multiprocess.py)

Esta versão contorna a limitação do GIL utilizando processos independentes (multiprocessing), cada um com seu próprio interpretador e espaço de memória. O domínio é decomposto em blocos contíguos de colunas (split_columns), distribuindo o resto da divisão W mod n_tasks entre os primeiros blocos para equilibrar a carga. Cada processo trabalhador (worker) mantém localmente apenas seu bloco de colunas, o que, ao contrário da versão multithread, exige comunicação explícita entre processos vizinhos para obter as colunas de fronteira (ghost columns) necessárias ao cálculo da vizinhança nas bordas do bloco.

A topologia de comunicação é estabelecida por um anel de Pipes bidirecionais: cada processo mantém uma conexão com o vizinho à esquerda e outra com o vizinho à direita, formando uma cadeia circular fechada consistente com a topologia toroidal do

tabuleiro (o último bloco se comunica com o primeiro). A cada geração, antes do cálculo, cada processo envia sua coluna mais à esquerda e mais à direita aos respectivos vizinhos e recebe as colunas de fronteira correspondentes (left_ghost, right_ghost) — um caso particular é tratado quando n_tasks=1, em que o próprio bloco fornece suas colunas fantasma via indexação toroidal direta, sem uso de Pipe. O cálculo da vizinhança em cada célula do bloco consulta a matriz local para colunas internas e as colunas fantasma recebidas para colunas na fronteira do bloco.

Ao término das max_iter + 1 gerações, cada processo soma localmente as células vivas de seu bloco — equivalente a uma cópia privada de acumulador, no espírito de uma redução OpenMP — e envia o total parcial ao processo principal por meio de um Pipe dedicado exclusivamente à contagem (count_conn), desacoplado do Pipe originalmente reservado à extração periódica do tabuleiro completo para geração de arquivos PBM. O processo principal recebe as somas parciais de todos os trabalhadores antes de sincronizar o encerramento (join), evitando bloqueios por saturação do buffer dos Pipes. O custo principal desta abordagem é a serialização (pickling) das colunas de fronteira a cada geração para transmissão entre processos, repetida n_tasks vezes por passo de simulação.

3.8. Comparação das Estratégias de Paralelização

O Quadro 1 resume as principais diferenças estruturais entre as implementações.

Tabela 1. Comparação das estratégias de decomposição e sincronização

Implementação	Linguagem	Unidade	Decomposição	Sincronização	Memória
Serial	C	-	Nenhuma	-	Única
OpenMP	C	Thread	Colunas	Barreira	Compart.
Serial	Python	-	Nenhuma	-	Única
Multithread	Python	Thread	Colunas	Future	Compart.

Implementação	Linguagem	Unidade	Decomposição	Sincronização	Memória
MP Pipe	Python	Processo	Colunas	Pipe	Isolada

Em todas as versões paralelas, seja em C ou em Python, a decomposição do domínio segue o mesmo princípio unidimensional por faixas contíguas de colunas, o que preserva a comparabilidade dos resultados entre linguagens e modelos de execução: as diferenças de desempenho observadas na Seção 4 podem ser atribuídas ao mecanismo de sincronização, ao modelo de memória e ao custo de execução da linguagem empregados, e não a diferenças na estratégia particionamento do problema ou na lógica das regras de transição, que é idêntica em todas as implementações.

3.9. Arquitetura das CPUs utilizadas

- **CPU 1:** AMD Ryzen 5 7520U (4 Núcleos / 8 Threads)
    - Memória Cache: L1: 256 KiB / L2: 2 MiB / L3: 4 MiB
    - Memória RAM: 16 GB (2x 8GB) LPDDR5 Samsung @ 5500 MT/s (Dual-Channel)
    - SO: Ubuntu 26.04 / Kernel: 7.0.0-27-generic
- **CPU 2:** Intel Core i5-1135G7 (4 Núcleos / 8 Threads)
    - Memória Cache: L1: 320 KiB / L2: 5MB / L3: 8MB
    - Memória RAM: 12 GB DDR4 2667 MT/s
    - SO: Ubuntu 26.04 / Kernel: 7.0.0-29-generic

4. Resultados

4.1. Análise Comparativa de Arquiteturas (CPU 1 vs. CPU 2)

A execução da simulação em dois ambientes de hardware distintos — a CPU 1 (AMD Ryzen 5 com memória LPDDR5 a 5500 MT/s) e a CPU 2 (Intel Core i5 com memória DDR4) — evidencia como as características físicas de cada arquitetura reagem aos diferentes modelos de programação.

4.1.1. Desempenho Bruto e Hierarquia de Cache (C e OpenMP)

Nas implementações nativas em C, a CPU 2 obteve vantagem expressiva. Na versão serial, concluiu em 22,10s contra 50,71s da CPU 1; com OpenMP (4 threads), registrou 7,90s frente a 13,18s. Essa superioridade deriva da maior capacidade de cache L3 da arquitetura Intel, otimizada para footprints de memória entre 4 MB e 8 MB. Como a paralelização por colunas destrói a localidade espacial em matrizes row-major, um último nível de cache (LLC) mais robusto amortece a penalidade de acesso à RAM e assegura uma maior taxa de Instruções por Ciclo (IPC).

4.1.2. Banda de Memória e Serialização (Python Multiprocessing)

Nas abordagens de múltiplos processos em Python, o cenário se inverte. Na versão via Pipe com 4 processos, a CPU 1 assumiu a dianteira e finalizou em 914s (wall-clock total extraído), superando os limites da CPU 2 (que possui DDR4). A comunicação por Pipe exige constante serialização e desserialização (pickling) das fronteiras da matriz. Neste cenário altamente dependente da largura de banda, a tecnologia LPDDR5 a 5500 MT/s confere à arquitetura Ryzen uma superioridade decisiva sobre a DDR4 no tráfego dos dados.

4.1.3. Limitações do SMT (Simultaneous Multithreading)

Ambos os processadores operam com 4 núcleos físicos e 8 threads lógicas. Os dados demonstram que o speedup S(p) cresce linearmente até p=4. Contudo, a transição para p=8 provoca uma queda acentuada na eficiência global E(p). Durante a execução, as threads lógicas virtuais competem intensamente pelas mesmas Unidades Lógicas e Aritméticas (ALUs) e pelas mesmas caches L1/L2 dos núcleos físicos. Consequentemente, o SMT entrega ganhos computacionais marginais, o que justifica a degradação da curva de escalabilidade de ambas as máquinas ao se atingir a lotação máxima de threads.

Tabela 2. Comparativo de Tempo e Recursos - CPU 1

Versão	Wall-clock	% CPU	RAM	Page Faults	Ctx Switches
C Serial	50.71s	99%	3.752	0/569	1/1.371

Versão	Wall-clock	% CPU	RAM	Page Faults	Ctx Switches
C OpenMP	13.18s	399%	4.112	4/584	6/345
Py Serial	20m 59s	99%	13.960	0/2.397.447	2/19.752
Py Thread (4T)	22m 25s	100%	18.600	2/3.611	1.32M/21.841
Py MP Pipe (4P)	914.75s	319%	26.896	0/944.080	415k/55k

Tabela 3. Comparativo de Tempo e Recursos - CPU 2

Versão	Wall-clock	% CPU	RAM	Ctx Switches
C Serial	22.10s	99%	3.960	1/709
C OpenMP	7.90s	399%	4.252	1/112
Py Serial	20m 01s	99%	13.992	1/10.048
Py Thread (4T)	19m 13s	100%	18.852	884k/7.2k
Py MP Pipe (4P)	4m 59s	387%	19.140	16k/5.7k

4.2. Análise de Comportamento: CPU-bound vs. I/O-bound

Para determinar se uma aplicação possui comportamento predominantemente voltado ao processamento (CPU-bound) ou a operações de entrada e saída (I/O-bound), analisam-se conjuntamente o percentual de utilização da CPU, a proporção entre User time e System time, além da incidência de falhas de página (page faults) e operações de leitura/escrita em disco.

Com base nos dados extraídos através do comando /usr/bin/time -v, todas as implementações avaliadas caracterizam-se estritamente como CPU-bound, embora

apresentem nuances e penalidades de desempenho distintas derivadas de suas arquiteturas de execução.

Justificativas por Versão

1. C Serial e C OpenMP (4 Threads): Comportamento CPU-bound puro. Ambas as versões apresentam um tempo de execução em modo kernel (System time) praticamente desprezível e um percentual de utilização da CPU que satura os recursos disponíveis (99% na versão serial e 399% na versão paralela com 4 núcleos). O consumo de recursos é inteiramente direcionado ao cálculo matemático intensivo da Vizinhança de Moore sobre a matriz toroidal, sem esperas por I/O de disco ou rede.

2. Python Serial: Comportamento CPU-bound (com sobrecarga de interpretação). O tempo em modo utilizador (User time) domina a execução, enquanto o tempo de kernel permanece irrelevante. O uso da CPU mantém-se fixo em 99%. O fator limitante não é o acesso a dispositivos externos, mas sim o custo computacional imposto pelo interpretador CPython ao processar os laços aninhados e gerir objetos dinâmicos no heap.

3. Python Multithreading (ThreadPoolExecutor): Comportamento CPU-bound (travado por concorrência / Lock-bound). Embora o programa utilize múltiplas threads para computar blocos do tabuleiro, o uso global da CPU permanece limitado a ~100%. O tempo de execução é consumido quase na totalidade em espaço de utilizador para gerir a concorrência. A aplicação sofre de intensa contenção interna devido ao Global Interpreter Lock (GIL) e a milhões de trocas de contexto voluntárias, caracterizando uma carga estritamente voltada ao processamento, mas severamente estrangulada pelas travas de sincronização da linguagem.

4. Python Multiprocessing (Pipe): Comportamento CPU-bound com forte penalidade de comunicação (Communication-bound). Registra percentuais de uso de CPU elevados (próximos a 380% com 4 processos) e System times baixos. Contudo, a variante baseada em Pipes introduz um custo computacional massivo de serialização (pickling) e desserialização de dados a cada geração para atualizar as colunas de fronteira. Embora o perfil fundamental permaneça centrado no esforço da CPU, o desempenho degrada consideravelmente devido ao tráfego de dados entre os espaços de memória isolados dos processos, aproximando-se de um comportamento restrito por largura de banda (communication-bound).

4.3. Análise Comparativa: C vs. Python (Abordagem Serial)

4.3.1. Paradigmas de Execução: Linguagem Compilada vs. Interpretada

A disparidade abissal de desempenho observada entre as implementações seriais reflete, fundamentalmente, a diferença arquitetural entre os paradigmas de execução de linguagens compiladas e linguagens interpretadas. O C é uma linguagem de tipagem estática e nativamente compilada (ahead-of-time). Durante o processamento do Jogo da Vida, suas instruções binárias atuam diretamente sobre a memória via aritmética de ponteiros, extraindo máxima vazão computacional.

Em contrapartida, o CPython atua como um interpretador de bytecode dinâmico. Para uma simulação de matriz densa que realiza milhões de varreduras em laços aninhados, esse modelo introduz um overhead massivo, pois cada acesso celular exige verificações de tipo em tempo real e chamadas ao gerenciador de memória (garbage collector).

Tabela 4. Contadores de Desempenho Físico e Profiling (Versão C Serial)

Métrica Exigida	CPU 1 (AMD Ryzen 5)	CPU 2 (Intel Core i5)
Ciclos Totais	212.4 Bilhões	91.36 Bilhões
Instruções	161,2 Bilhões	160,94 Bilhões
IPC (Vazão)	0,76	1,76
Cache-misses	136,3 Milhões	322 Milhões (L1)
Branch-misses	56,3 Milhões	0,25%

4.3.2. Comprovação Empírica pelo Profiling de Hardware (perf stat)

A comparação dos dados do perf stat entre as versões seriais da CPU 1 evidencia o abismo entre as duas linguagens computando o mesmo problema. A versão C concluiu a execução em aproximadamente 50 segundos, consumindo 212 bilhões de ciclos da CPU e executando 161 bilhões de instruções reais de máquina.

Para realizar a exata mesma tarefa e entregar a mesma matriz de saída, o interpretador Python demorou quase 21 minutos, o que demandou absurdos 5,3 trilhões de ciclos da CPU e 17,5 trilhões de instruções. Isso significa que, para cada

instrução útil que o programa em C executou, o Python precisou executar mais de 100 instruções de "gerenciamento" interno de objetos (overhead do interpretador) apenas para viabilizar a lógica matemática da Vizinhança de Moore.

Além do volume de instruções, o paradigma de objetos do Python destruiu a localidade de referência na hierarquia de memória. Enquanto a matriz em C sofreu cerca de 136 milhões de faltas de cache (cache-misses) totais, as listas nativas aninhadas do Python (list of lists) registraram 408 milhões de cache-misses e impressionantes 4,4 bilhões de faltas na cache L1 de dados (L1-dcache-load-misses). A CPU gastou grande parcela de seus ciclos travada aguardando o processamento de referências de objetos Python espalhados desordenadamente pela RAM.

4.3.3. Comportamento do Sistema Operacional (strace e Hotspots)

A análise do comportamento sistêmico confirma que as duas linguagens penalizaram o hardware de formas distintas. Através do gprof, observa-se que no C 100% do tempo de execução concentrou-se estritamente na função matemática de processamento da grade (evolve). As interações com o sistema operacional (strace) foram insignificantes: apenas 8 chamadas de alocação genérica (mmap) que custaram singelos 75 microssegundos no tempo total.

Por outro lado, o CPython precisou escalar agressivamente os recursos dinâmicos. O strace registrou mais de 80.000 chamadas à função brk — uma requisição direta ao Kernel do Linux para expandir ou contrair o limite de espaço de memória do processo (heap limit). Essas alocações constantes ocorrem porque, ao reescrever a matriz em cada uma das 5000 iterações na função evolve (que reteve 1577 segundos do cProfile), o Python destrói os números inteiros antigos e instancia milhares de novos objetos do tipo int a cada etapa de evolução, sobrecarregando tanto a ALU do processador quanto as tabelas de gerenciamento de memória do Kernel.

Análise C Serial

Tabela 5. C Serial: Métricas de Hardware via perf stat (CPU 1)

Métrica	Valor	% Eventos
Cycles	212.423.047.978	71.42%

Métrica	Valor	% Eventos
Instructions	161.218.929.814	71.43%
Cache-misses	136.351.011	71.43%
L1-dcache-miss	350.449.855	71.43%
Branch-misses	56.290.608	71.43%

Tabela 6. C Serial: Análise de Hotspots via gprof (CPU 1)

% Time	Self (s)	Calls	Tot/Call	Função
100.00	49.99	5.001	0.01	evolve

Tabela 7. C Serial: Faltas de Cache por Instrução via Callgrind (CPU 1)

Função	Ir	Dr	Dw	D1mr	D1mw
evolve	160,1Bi	17,5Bi	2,5Bi	78,3Mi	79,3Mi

Tabela 8. C Serial: Top 3 Chamadas de Sistema via strace (CPU 1)

% Time	Segundos	µs / Call	Calls	Syscall
13.04	0.000075	9	8	mmap
1.91	0.000011	3	3	fstat
0.70	0.000004	1	3	brk

Tabela 9. C Serial: Métricas de Hardware via perf stat (CPU 2)

Métrica	Valor	% Eventos
Cycles	91.783.122.923	-
Instructions	160.957.891.471	-
Cache-misses	921.887	-

Métrica	Valor	% Eventos
L1-dcache-miss	323.344.539	-
Branch-misses	87.288.195	-

Tabela 10. C Serial: Análise de Hotspots via gprof (CPU 2)

% Time	Self (s)	Calls	Tot/Call	Função
100.00	22.08	5.001	0.004	evolve

Tabela 11. C Serial: Top Chamadas de Sistema via strace (CPU 2)

% Time	Segundos	µs / Call	Calls	Syscall
0,00	0.000000	0	8	mmap
0,00	0.000000	0	3	fstat

Análise Python Serial

Tabela 12. Python Serial: Métricas de Hardware via perf stat (CPU 1)

Métrica	Valor	% Eventos
Cycles	5.313.323.064.534	71.43%
Instructions	17.591.544.342.430	71.43%
Cache-misses	408.146.708	71.43%
L1-dcache-miss	4.491.154.916	71.43%

Tabela 13. Python Serial: Top Hotspots Nativos via cProfile (CPU 1)

ncalls	tottime	percall	cumtime	Função
5001	1577.275	0.315	1577.275	gol.py(evolve)
1	2.207	2.207	1579.483	gol.py(game)

Tabela 14. Python Serial: Top 3 Chamadas de Sistema via strace (CPU 1)

% Time	Segundos	µs / Call	Calls	Syscall
99.32	0.422889	5	80.049	brk
0.03	0.000119	1	78	newfstatat

Tabela 15. Python Serial: Métricas de Hardware via perf stat (CPU 2)

Métrica	Valor	% Eventos
Cycles	4.460.144.195.537	-
Instructions	17.583.597.025.135	-
Cache-misses	160.778.323	-
L1-dcache-miss	2.184.631.801	-

Tabela 16. Python Serial: Top Hotspots Nativos via cProfile (CPU 2)

ncalls	tottime	percall	cumtime	Função
5001	1472.690	0.294	1472.690	gol.py(evolve)

Tabela 17. Python Serial: Top 3 Chamadas de Sistema via strace (CPU 2)

% Time	Segundos	µs / Call	Calls	Syscall
98.93	0.064318	0	80.045	brk

4.4. Multithreading vs. Multiprocessing e o Impacto do GIL

Enquanto em C (via OpenMP) o multithreading é a abordagem padrão para paralelismo, no Python (CPython) essa estratégia esbarra no GIL (Global Interpreter Lock). O GIL é uma trava de exclusão mútua (mutex) que permite que apenas uma única thread execute bytecodes por vez, protegendo o gerenciamento automático de memória da linguagem contra condições de corrida.

Para algoritmos estritamente CPU-bound como o Jogo da Vida, o GIL anula o paralelismo real. Mesmo com múltiplas threads instanciadas, elas não calculam a matriz simultaneamente nos núcleos da CPU; apenas revezam a posse da trava. O

profiling comprova este gargalo: o sistema demonstra características de lock-bound, perdendo tempo massivo em chamadas de bloqueio (acquire_lock, sem_wait) e trocas de contexto, o que frequentemente resulta em um desempenho inferior ao da versão serial.

A Tabela 18 evidencia empiricamente a serialização imposta pelo GIL. O somatório de quatro chamadas espelha quase perfeitamente o tempo gasto na variante serial. As funções de espera, como Thread.join e Condition.wait, acumulam milhares de segundos, indicando extrema ociosidade das threads.

Tabela 18. Python Multithreading: Principais Funções de Sincronização via cProfile (CPU 1)

Função	Chamadas	Tottime (s)	Cumtime (s)
Thread.join	4	0,000	1664,172
Lock.acquire	81449	14,779	16,437
Condition.wait	20707	0,373	3354,980
Future.result	4001	20,167	3362,876

Para contornar o GIL e explorar múltiplos núcleos, o Python exige a adoção do Multiprocessing. Essa abordagem cria processos independentes no nível do sistema operacional, cada qual com seu próprio interpretador, espaço de memória e seu próprio GIL. O profiling atesta que o custo inicial de instanciação (posix.fork e criação do Pipe) é irrisório, na casa dos 7 milissegundos. Adicionalmente, por utilizar Process e fork nativos em substituição ao Pool.map, o algoritmo contorna o custo pesado da serialização inicial (pickle.dumps).

Tabela 19. Python Multiprocessing: Custo de Criação e Comunicação (Pai) CPU 1

Função	Chamadas	Cumtime (s)
Process.start	4	0,007
posix.fork	4	0,005

Função	Chamadas	Cumtime (s)
Pipe ()	12	0,013
Conn.recv	4	203,074
marshal.loads	30	0,004

(Nota: O tempo cumulativo extremo de Connection.recv engloba todo o tempo bloqueante de espera do processo mestre).

Tabela 20. Python Multiprocessing: Custo de Criação e Comunicação (Pai) CPU 2

Função	Chamadas	Cumtime (s)
Process.start	4	0,010
posix.fork	4	0,009
Pipe ()	12	0,020
Conn.recv	4	257,458
marshal.loads	30	0,003

O gargalo deixa de ser a criação de instâncias e passa a ser o alto custo da Comunicação Interprocessos (IPC). Conforme a Tabela 21, a versão via Pipe gasta quase a totalidade do seu tempo ocioso em Connection.recv (mais de 200 segundos), demonstrando forte característica de restrição por comunicação (communication-bound).

Tabela 21. Python Multiprocessing: Gargalos de Comunicação e Sincronização (CPU 1)

Função	Chamadas	Cumtime (s)	% Ocioso
Conn.recv	40.000	203,074	99.98%
posix.read	8	203,073	99.98%

Tabela 22. Perfil de execução das principais funções do Python Multithreading

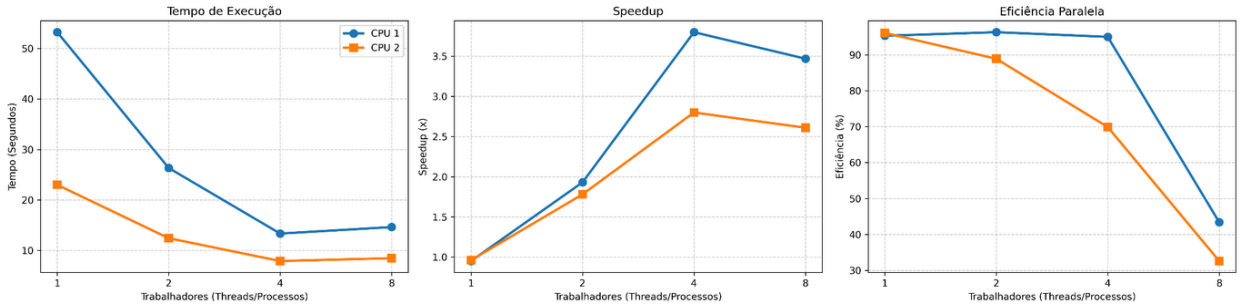
Função	T. Excl (1)	T. Cum (1)	T. Excl (2)	T. Cum (2)
base.result	20,167s	3.362s	0,092s	2.308s
thread.wait	0,373s	3.354s	0,256s	2.305s
thread.join	0,000s	1.664s	0,000s	1.405s
lock.acquire	14.779s	16,43s	36,053s	36,49s

4.5. Multiprocessing: Comunicação via Pipe

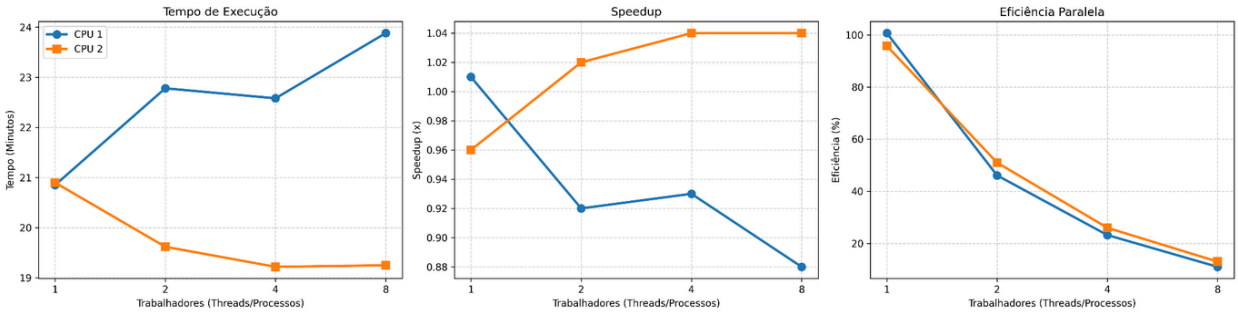
Ao contornar o GIL por meio de processos isolados, o desafio do paralelismo em Python desloca-se para a forma como esses processos sincronizam os dados através da troca de mensagens (Pipes). Para atualizar as "colunas fantasmas", o Python é obrigado a serializar e desserializar (pickling) suas estruturas de dados nativas a cada geração simulada. Esse ciclo constante de empacotamento impõe um overhead massivo de cópia de dados via Sistema Operacional, transformando o gargalo da aplicação, que deixa de focar na lógica celular (CPU-bound) e passa a ser estrangulada pelo tráfego de dados (I/O-bound). A tentativa de registrar a totalidade dessas trocas de contexto pelo perf stat causou o esgotamento de memória (Out of Memory).

4.6. Análise de Escalabilidade e Eficiência Paralela

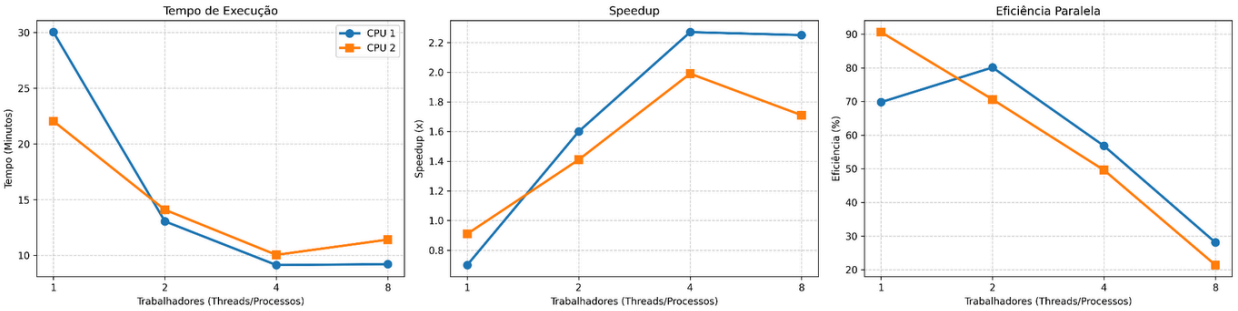
Escalabilidade do C Paralelizado com OpenMP.



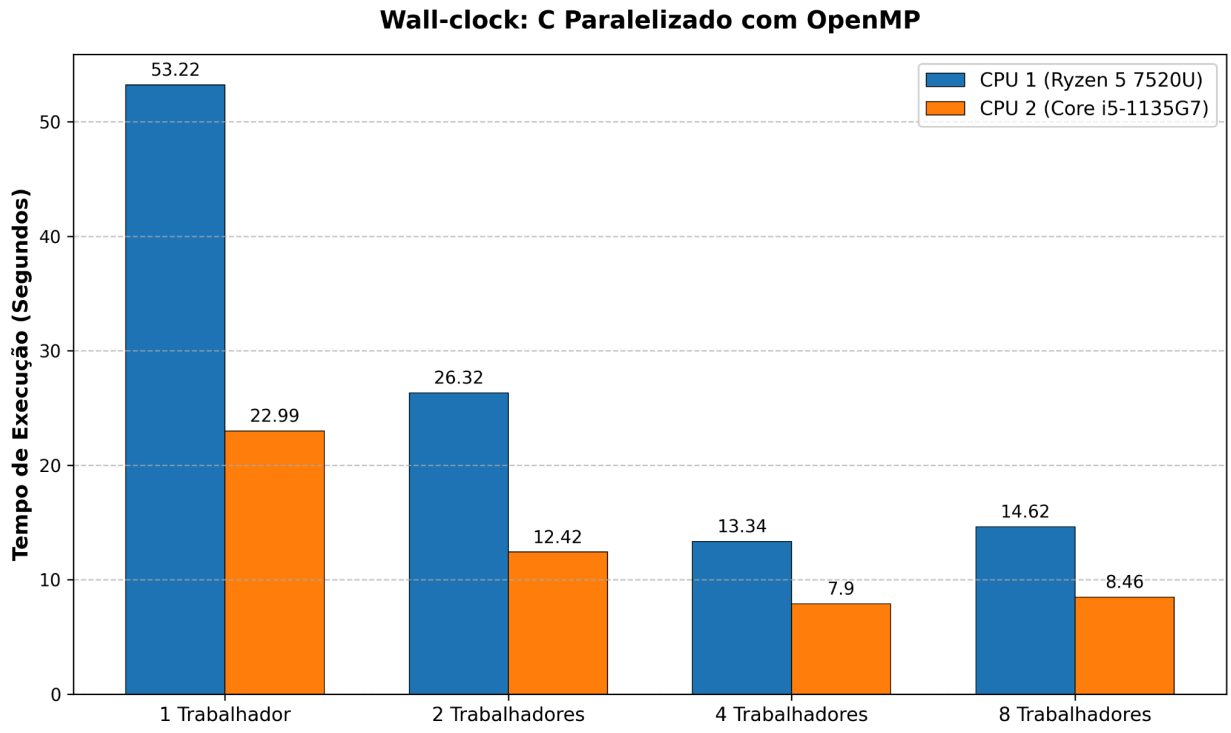
Escalabilidade do Python Paralelo com Multithreading (ThreadPoolExecutor).

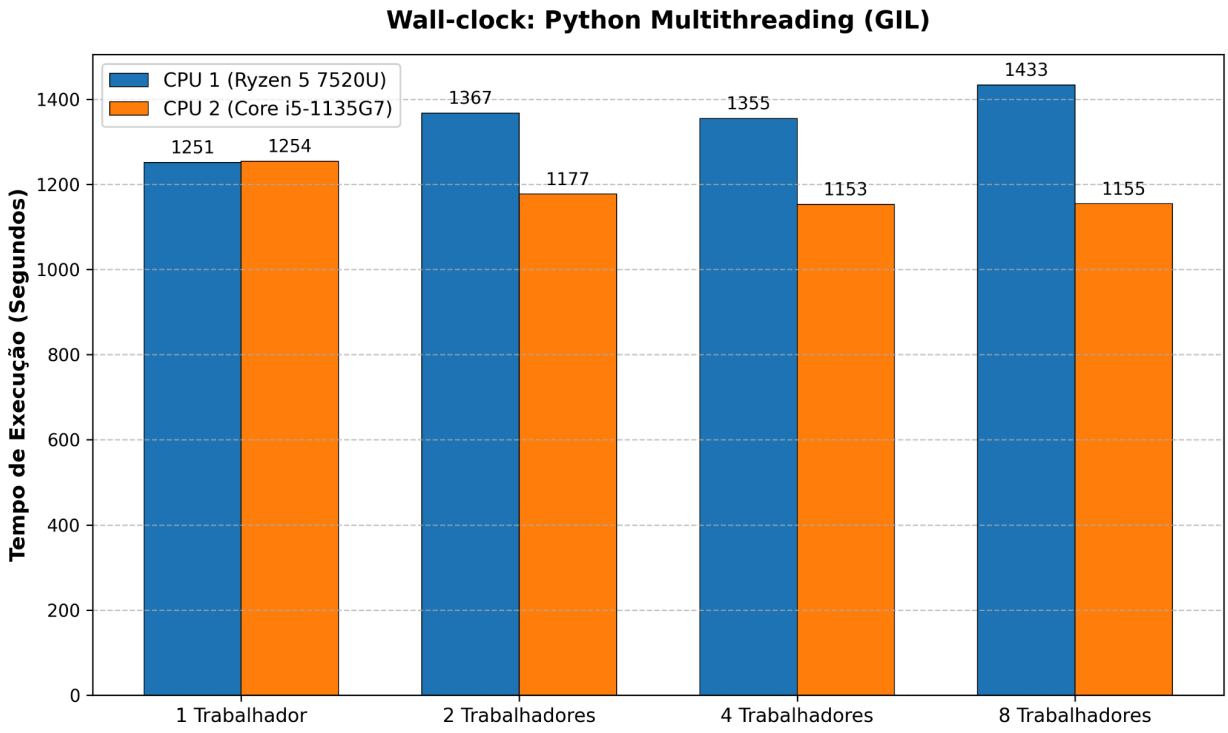


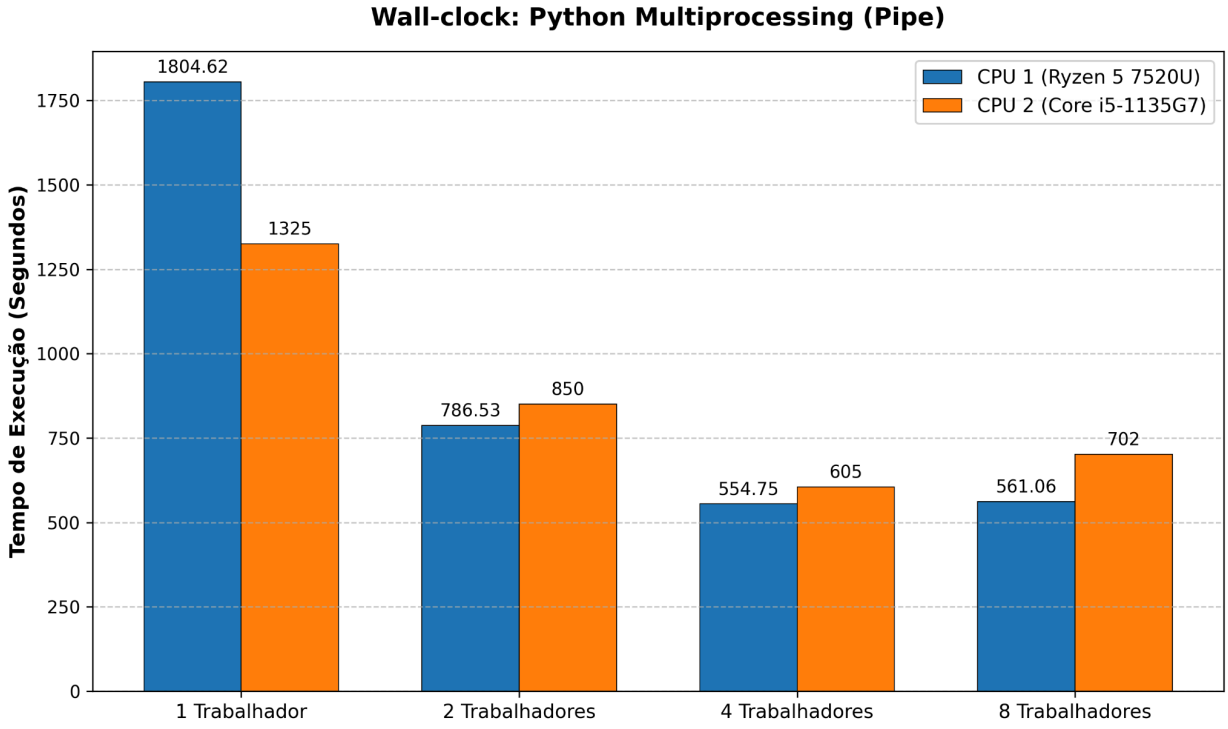
Escalabilidade do Python Paralelo com Multiprocessing utilizando Pipe.

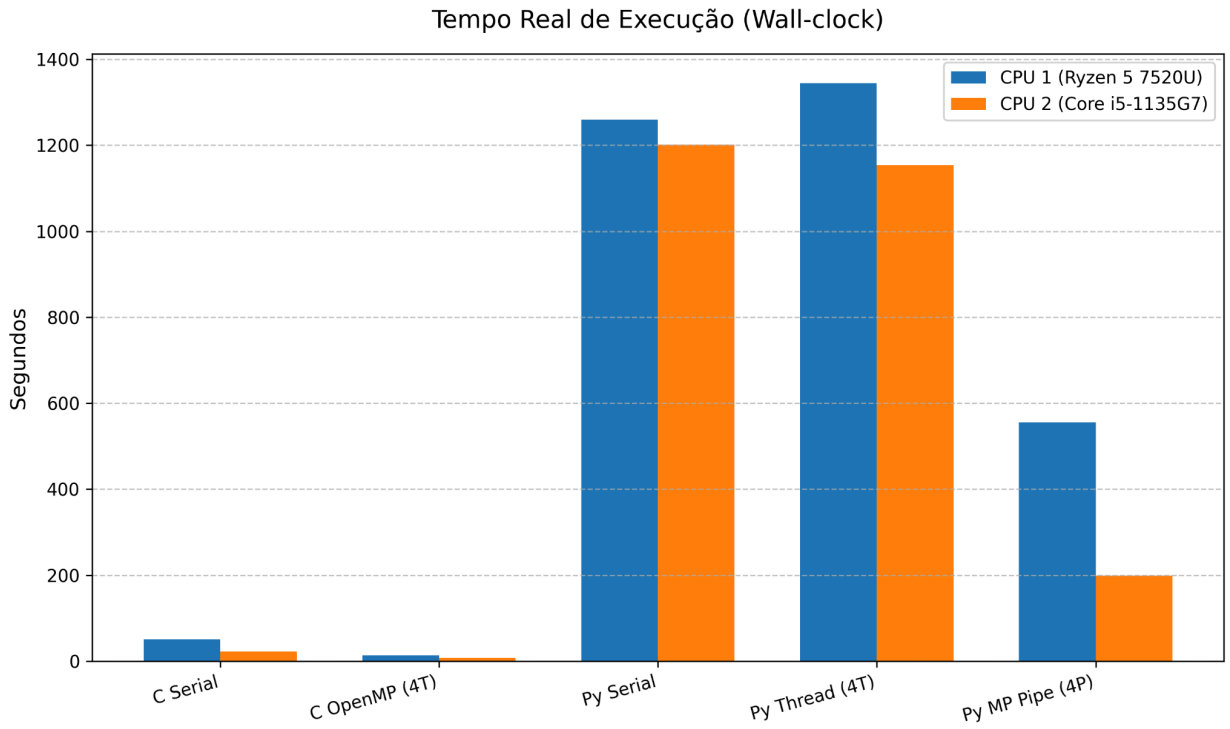


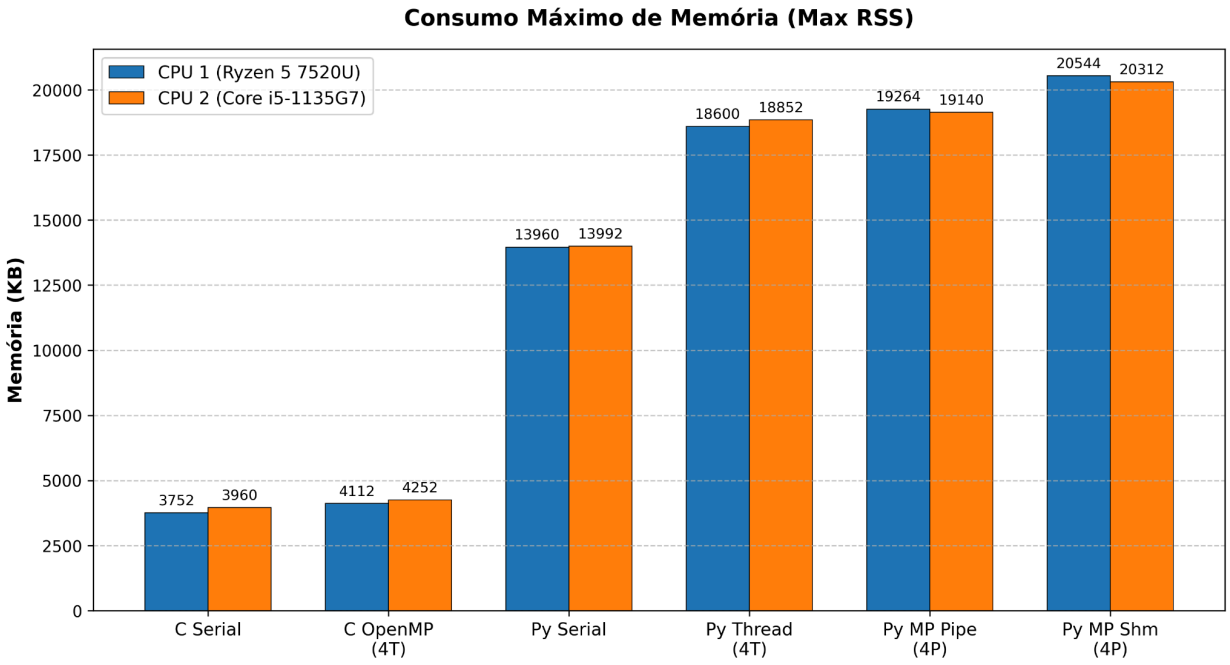
4.7. Comparação Geral dos Modelos de Paralelização

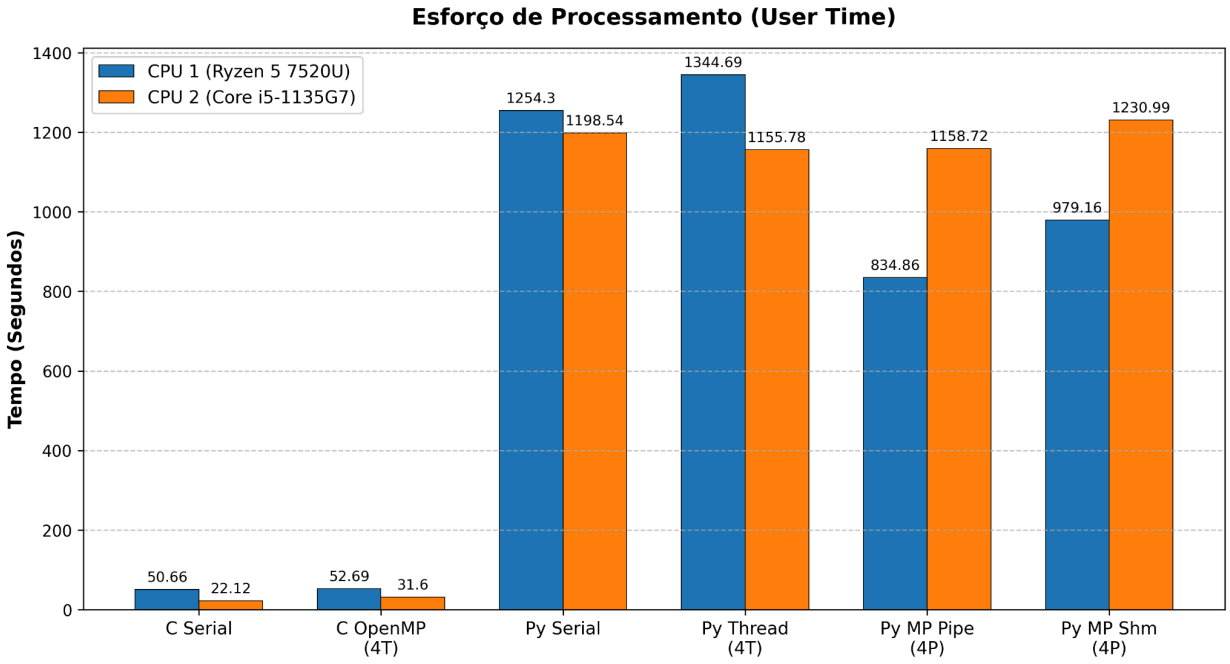


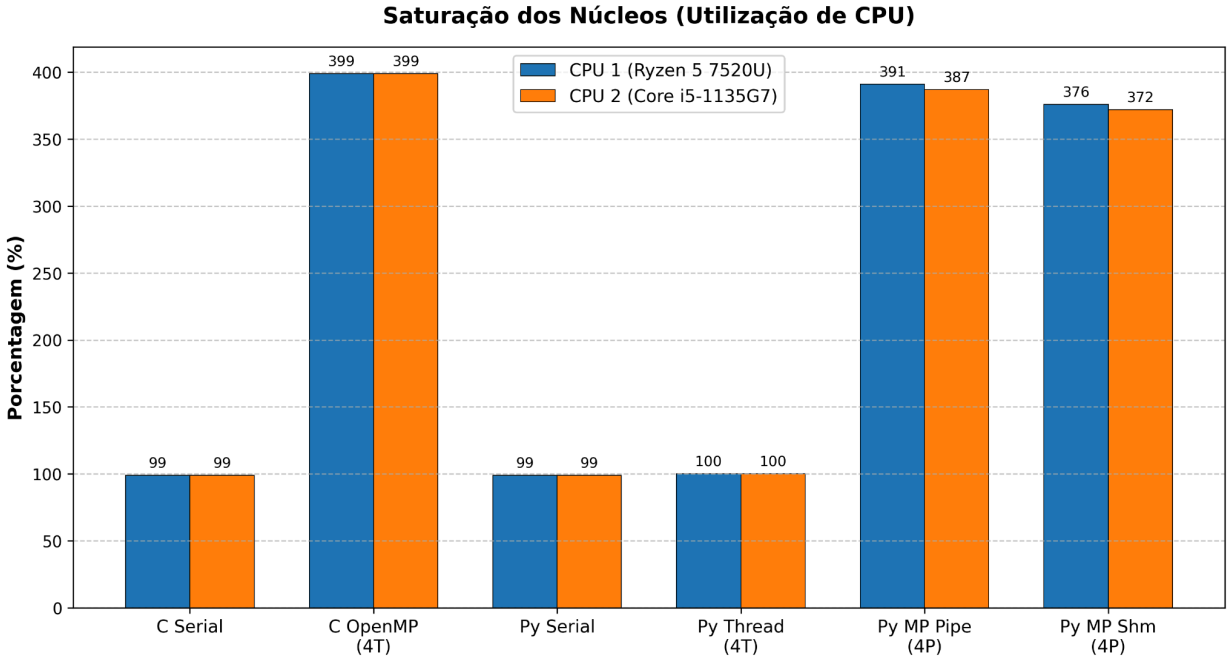

Wall-clock: Python Multithreading (GIL)
CPU 1 (Ryzen 5 7520U)
CPU 2 (Core i5-1135G7)
Tempo de Execução (Segundos)
0
200
400
600
800
1000
1200
1400
1251
1254
1367
1177
1355
1153
1433
1155
1 Trabalhador
2 Trabalhadores
4 Trabalhadores
8 Trabalhadores

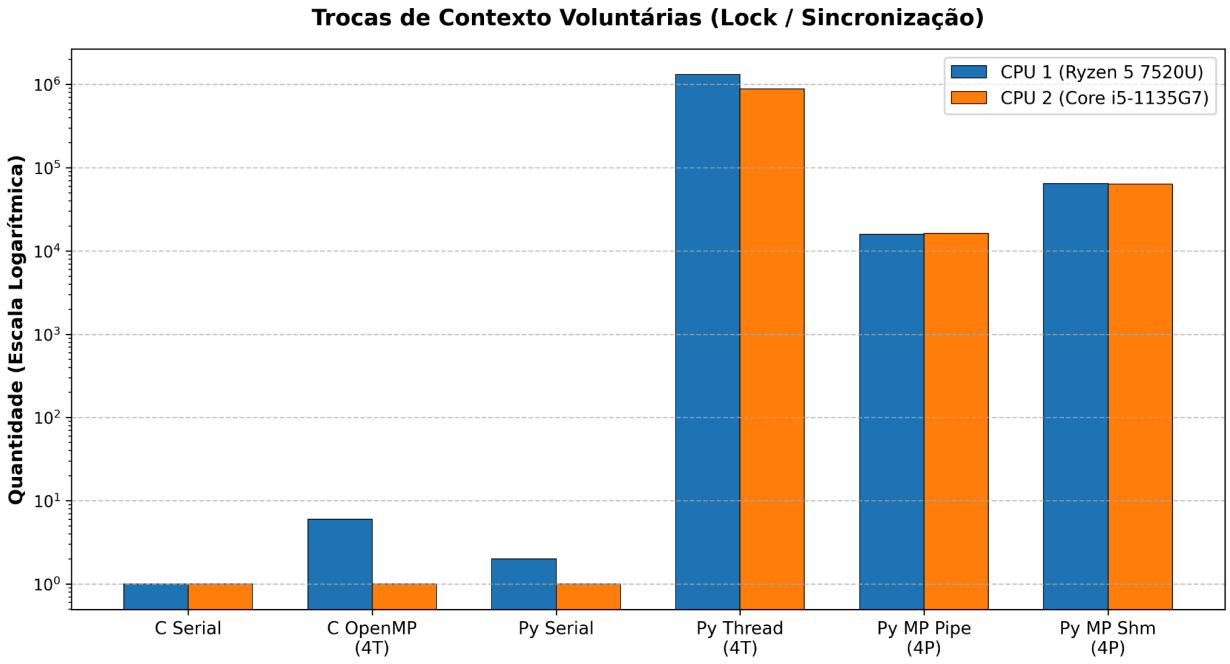

Wall-clock: Python Multiprocessing (Pipe)
CPU 1 (Ryzen 5 7520U)
CPU 2 (Core i5-1135G7)
Tempo de Execução (Segundos)
0
250
500
750
1000
1250
1500
1750
1804.62
1325
786.53
850
554.75
605
561.06
702
1 Trabalhador
2 Trabalhadores
4 Trabalhadores
8 Trabalhadores


Tempo Real de Execução (Wall-clock)
Segundos
CPU 1 (Ryzen 5 7520U)
CPU 2 (Core i5-1135G7)
0
200
400
600
800
1000
1200
1400
C Serial
C OpenMP (4T)
Py Serial
Py Thread (4T)
Py MP Pipe (4P)


Consumo Máximo de Memória (Max RSS)
Memória (KB)
CPU 1 (Ryzen 5 7520U)
CPU 2 (Core i5-1135G7)
0
2500
5000
7500
10000
12500
15000
17500
20000
3752
3960
4112
4252
13960
13992
18600
18852
19264
19140
20544
20312
C Serial
C OpenMP (4T)
Py Serial
Py Thread (4T)
Py MP Pipe (4P)
Py MP Shm (4P)


Esforço de Processamento (User Time)
CPU 1 (Ryzen 5 7520U)
CPU 2 (Core i5-1135G7)
Tempo (Segundos)
0
200
400
600
800
1000
1200
1400
50.66
22.12
52.69
31.6
1254.3
1198.54
1344.69
1155.78
834.86
1158.72
979.16
1230.99
C Serial
C OpenMP (4T)
Py Serial
Py Thread (4T)
Py MP Pipe (4P)
Py MP Shm (4P)


Saturação dos Núcleos (Utilização de CPU)
CPU 1 (Ryzen 5 7520U)
CPU 2 (Core i5-1135G7)
Porcentagem (%)
0
50
100
150
200
250
300
350
400
99
99
399
399
99
99
100
100
391
387
376
372
C Serial
C OpenMP (4T)
Py Serial
Py Thread (4T)
Py MP Pipe (4P)
Py MP Shm (4P)



5. Conclusão

A alta independência computacional entre as células do autômato durante a evolução justifica diretamente a viabilidade da paralelização entre múltiplos núcleos. A abordagem mitiga expressivamente o tempo total e otimiza a capacidade de processamento dos componentes de hardware. Entretanto, a versão em Python demonstrou possuir um custo computacional massivo que expôs o elevado overhead característico da linguagem interpretada. Em contrapartida, linguagens compiladas (como C) maximizam a eficiência executando instruções nativas diretamente no hardware, garantindo a maximização da eficiência ao efetuar a execução das instruções paralelizadas diretamente na estrutura física de memória.